

Міністерство освіти і науки України
НТУУ «КПІ ім. Ігоря Сікорського»
Навчально-науковий інститут атомної та теплової енергетики
Кафедра цифрових технологій в енергетиці

Розрахункова графічна робота
з дисципліни
«Візуалізація графічної та геометричної інформації»
Тема «Операції з текстурними координатами»
Варіант №15

Студентки 1-го курсу НН ІАТЕ гр. ТР-51мп
Імаської Тетяни
к.н.т., професор Демчишин А.А.

КИЇВ 2025

Мета роботи: метою розрахунково-графічної роботи є розроблення та реалізація програмного модуля системи текстурного відображення у середовищі WebGL, що забезпечує виконання афінних перетворень текстури, зокрема масштабування та обертання, відносно динамічно визначеної опорної точки на поверхні тривимірного об'єкта. Положення цієї опорної точки (pivot) має встановлюватися та змінюватися користувачем у просторі текстурних координат (u, v) у процесі інтерактивної взаємодії з програмою та відображатися у реальному часі.

Завдання:

1. Повторно використати (reuse) базову систему текстурних координат із попередньої роботи без переобчислення UV-розгортки.
2. Реалізувати масштабування текстури у просторі (u, v) з можливістю динамічної зміни коефіцієнта scaling користувачем.
3. Реалізувати обертання текстури на довільний кут навколо опорної точки (pivot), яка визначається у текстурних координатах.
4. Забезпечити переміщення pivot за допомогою клавіш A, D, W, S.
5. Забезпечити відображення поточного значення pivot у користувацькому інтерфейсі програми.
6. Інтегрувати інструменти зміни параметрів scaling і rotation у графічний інтерфейс (слайдери та інформаційні елементи).
7. Забезпечити коректну взаємодію між CPU-частиною програми (JavaScript) та GPU-частиною (shader pipeline).
8. Продемонструвати коректність роботи програми на прикладі тривимірної моделі.

Виконання зазначених вимог дозволяє опанувати принципи текстурного відображення, роботу з UV-координатами, використання шейдерів у WebGL, а також реалізацію інтерактивних афінних перетворень над текстурними даними.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Текстури та UV-координати у комп'ютерній графіці

Текстурування є одним з ключових етапів формування зображень у тривимірній графіці. Його сутність полягає у відображенні двовимірного зразка (текстури) на поверхню тривимірного об'єкта. Щоб виконати це відображення, кожній вершині моделі призначаються текстурні координати (u, v) – параметри у діапазоні $[0, 1]$, що визначають відповідні точки на текстурному зображенні.

Координата u визначає позицію по горизонталі, v – по вертикалі текстури. Таким чином, UV-координати описують відображення між поверхнею моделі та текстурою. Під час рендерингу фрагментний шейдер інтерполює UV-значення між вершинами, дозволяючи отримати колір пікселя з текстурного зображення.

UV-розгортка не обов'язково відповідає реальній геометрії: вона задає параметризацію поверхні, яка є незалежною від метричних властивостей моделі. Завдяки цьому одна й та сама текстура може бути повторно використана протягом різних операцій – саме цей підхід реалізовано в даній роботі.

Афінні перетворення

Афінні перетворення – це клас лінійних перетворень простору, які зберігають паралельність прямих і порядок точок. До них належать:

- масштабування (scaling);
- поворот (rotation);
- перенесення (translation);
- віддзеркалення та зсуви.

У просторі текстурних координат (u, v) афінні перетворення застосовуються не до геометрії об'єкта, а до текстури, що дозволяє змінювати її вигляд без модифікації самих вершин. У загальному випадку афінне перетворення описується матрицею:

$$\begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_u \\ c & d & t_v \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

де коефіцієнти a, b, c, d формують лінійну частину перетворення (масштаб і поворот), а t_u, t_v - перенесення.

Для однорідного масштабування:

$$u' = \frac{u}{s}, \quad v' = \frac{v}{s}$$

Для повороту на кут θ :

$$\begin{bmatrix} u' \\ v' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} u \\ v \end{bmatrix}$$

Pivot – центр перетворення

У більшості графічних систем афінні перетворення виконуються щодо центру текстури (0.5, 0.5). Це не завжди зручно, адже користувач може захотіти обернути або масштабувати текстуру не від центру, а від довільної точки на поверхні моделі.

Точка, відносно якої виконується перетворення, називається *pivot*. Щоб повернути або масштабувати текстуру навколо *pivot*, координати текстури переносять у систему відліку з початком у цій точці:

$$uv = uv - pivot; \quad uv = R(uv); \quad uv = uv + pivot$$

де R – матриця повороту.

Цей механізм і реалізовано у роботі.

Конвеєр WebGL та роль шейдерів

WebGL реалізує графічний конвеєр GPU, який складається з послідовних програмних стадій. У даній роботі задіяні дві основні:

1. Vertex shader (вершинний шейдер) – отримує позиції вершин, їхні нормалі та UV-координати і передає їх у фрагментний шейдер.
2. Fragment shader (фрагментний шейдер) – виконує обчислення для кожного фрагмента (пікселя). Саме тут виконано трансформації UV за параметрами:

```
uv -= pivot;  
uv /= scale;  
uv = R * uv;  
uv += pivot.
```

Керування параметрами перетворення здійснюється через uniform-змінні, які передаються з JavaScript у шейдерну програму.

ДЕТАЛІ РЕАЛІЗАЦІЇ

Структура та принцип роботи модуля

Реалізація системи текстурних перетворень виконана у середовищі WebGL з використанням графічного конвеєра, у якому обчислення координат та освітлення здійснюються на графічному процесорі. Програмний модуль складається з HTML-сторінки з елементом <canvas> для рендерингу, JavaScript-коду для створення контексту WebGL та шейдерних програм, а також GLSL-шейдерів, що відповідають за обробку текстурних координат.

Текстура закріплюється за моделлю через UV-координати, пов'язані з вершинами об'єкта. На відміну від маніпулювання геометрією, у даній реалізації трансформації виконуються виключно над UV-координатами, тобто змінюється спосіб відображення текстури, а не форма моделі. Такий підхід зберігає геометричну цілісність сцени та не впливає на положення камери чи світлових джерел.

Управління параметрами трансформацій

Для керування текстурними перетвореннями використовується три параметри: опорна точка (pivot), коефіцієнт масштабування та кут обертання. Вони зберігаються у JavaScript як глобальні змінні та передаються в шейдер через uniform-параметри:

```
let texPivot = { u: 0.0, v: 0.0 };  
let texScale = 1.0;  
let texAngle = 0.0; // radians
```

Опорна точка є центром, відносно якого виконуються обертання та масштабування текстури. Її координати змінюються за допомогою клавіш A/D (уздовж осі u) та W/S (уздовж осі v):

```
addEventListener('keydown', e => {  
  const step = 0.02;  
  if (e.code === 'KeyA') texPivot.u -= step;  
  if (e.code === 'KeyD') texPivot.u += step;  
  if (e.code === 'KeyW') texPivot.v += step;  
  if (e.code === 'KeyS') texPivot.v -= step;  
});
```

Таким чином, користувач не лише задає значення pivot, але й динамічно змінює його положення у просторі UV, що є ключовою вимогою завдання.

Реалізація масштабування та обертання у шейдері

Афінні перетворення текстури виконуються у фрагментному шейдері GLSL. Обчислення відбуваються безпосередньо перед вибіркою кольору з текстури, що гарантує коректне відображення незалежно від складності моделі. Спершу UV-координати переводяться у локальну систему відносно pivot, після чого

застосовується масштабування та обертання, і координати повертаються до глобальної системи:

```
vec2 pivot = vec2(0.5) + u_texPivot;

uv -= pivot;          // зміщення в систему pivot
uv /= u_texScale;     // масштабування

float c = cos(u_texAngle);
float s = sin(u_texAngle);
uv = mat2(c, -s, s, c) * uv; // обертання

uv += pivot;          // повернення координат
```

Така схема забезпечує виконання трансформацій не навколо центру текстури, а навколо точки, обраної користувачем. Це дозволяє створювати ефекти локального обертання, наближення та віддалення текстури у будь-якій ділянці поверхні моделі.

Підсумок розділу

Запропонована реалізація забезпечує:

- виконання масштабування та обертання текстури без зміни геометрії об'єкта;
- визначення довільної опорної точки трансформацій у координатах (u, v);
- інтерактивне керування параметрами у реальному часі;
- виконання всіх обчислень на GPU, що гарантує стабільну продуктивність.

Таким чином, модуль повністю відповідає технічним вимогам завдання щодо реалізації афінних перетворень текстурних координат навколо користувацького pivot.

ІНСТРУКЦІЯ КОРИСТУВАЧА

Цей розділ пояснює порядок запуску програмного забезпечення, а також описує елементи керування та можливості взаємодії з модулем текстурних перетворень.

Запуск програми

На рисунку 1 можемо побачити зображення фігури та інтерактивне меню. Для того щоб все запрацювало, необхідно:

1. відкрити файл index.html у браузері, який підтримує WebGL (Google Chrome / Mozilla Firefox / Microsoft Edge);
2. дочекатися ініціалізації сцени: на екрані з'явиться тривимірний об'єкт Ding-dong із графічною панеллю керування (меню параметрів).

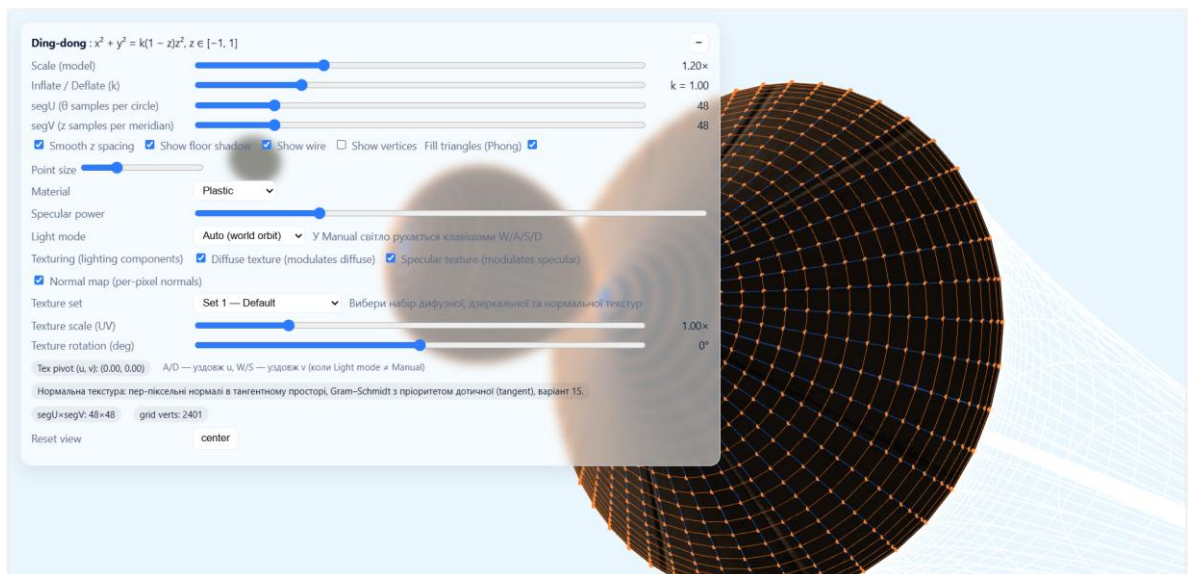


Рис. 1 — головний екран після запуску програми

Опис графічної панелі керування

Панель містить елементи, що дозволяють змінювати параметри сцени, освітлення та текстуризації. Користувач може змінювати масштаби моделі, матеріали, типи текстур та інші характеристики рендера.

Нас цікавлять ті елементи панелі(табл.1), що пов'язані з реалізацією поставленого завдання:

Елемент	Опис
Texture scale (UV)	Керує масштабуванням текстури у просторі UV
Texture rotation (deg)	Визначає кут повороту текстури навколо pivot
Tex pivot (u, v)	Поточні координати опорної точки трансформації
A/D, W/S	Переміщення pivot по осях u та v відповідно
Texture set	Вибір текстурних карт (дифузної, дзеркальної, нормалей)

Таблиця 1 – функціональні елементи інтерфейсу модуля текстуризації

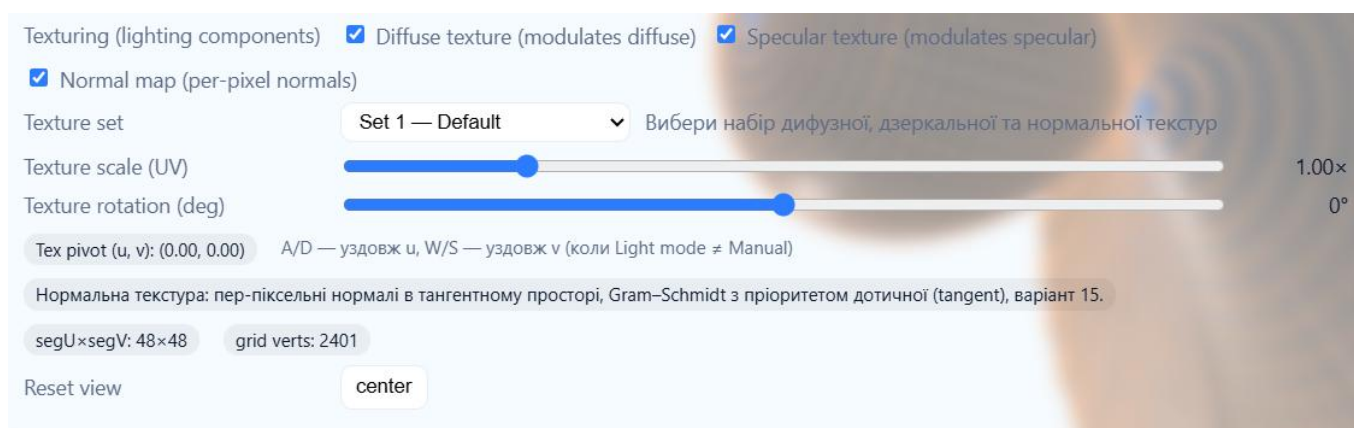


Рис. 2 – фрагмент меню з полями

Керування опорною точкою (pivot)

Опорна точка визначає, навколо якої обчислюються поворот і масштаб текстури. Її положення змінюється за допомогою клавіш клавіатури(табл.2):

Клавіша	Дія
A	зміщення pivot ліворуч ($u-$)
D	зміщення pivot праворуч ($u+$)
W	зміщення pivot вгору ($v+$)
S	зміщення pivot вниз ($v-$)

Таблиця 2 – клавіші керування опорною точкою (pivot) у просторі UV

Масштабування текстури

Масштабування змінює розмір текстури без зміни геометрії моделі:

- перемістити повзунок Texture scale (UV);
- значення >1.0 збільшує текстуру, <1.0 – зменшує.

Ефект застосовується відносно pivot, тому результат залежить від його положення. На рисунку 3 та 4 можна побачити різницю після зміни масштаба текстури.

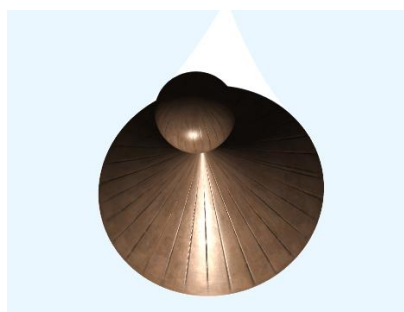


Рис.3 – зображена фігура з меншим масштабом

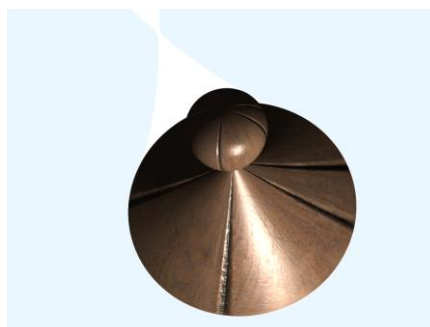


Рис.4 – зображена фігура з більшим масштабом

Поворот текстури навколо pivot

Поворот текстури виконується у градусах. Для цього:

- обрати точку pivot, змістивши її клавішами A/W/S/D.
- встановити значення у полі Texture rotation (deg).

При зміні значення куту текстура обертається навколо вибраної точки. На рисунку 5 можна побачити роботу Texture rotation.

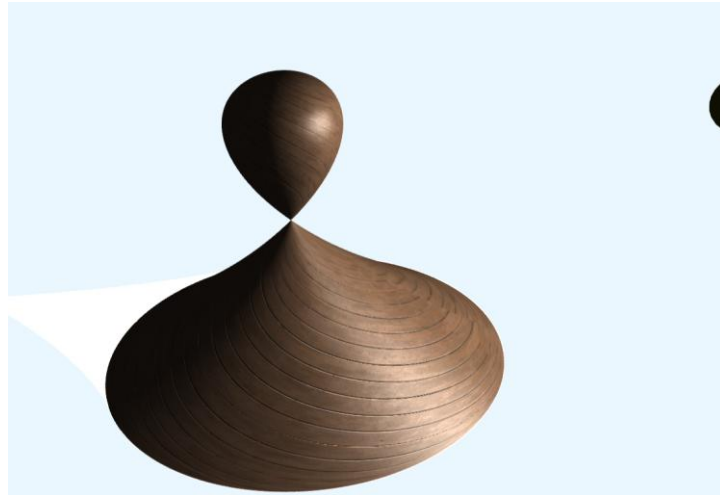


Рис. 5 – текстура після повороту

Особливості роботи системи

Трансформації виконуються над UV-координатами, а не над моделлю.

Геометрія об'єкта не змінюється – модифікується лише спосіб накладання текстури.

Опорна точка (pivot) є динамічною.

Вона не прив'язана до центру текстури і задається користувачем, змінюючись клавішами A/W/S/D у просторі (u, v).

Усі зміни застосовуються в реальному часі.

Оновлення трансформацій не потребує перезавантаження сцени або повторної ініціалізації рендера.

Масштабування та поворот можна комбінувати довільно. Обидві операції працюють незалежно й дають коректний результат у будь-якій послідовності.

Висновок

У даній розрахунково-графічній роботі було розроблено та реалізовано програмний модуль для виконання афінних перетворень текстурних координат у середовищі WebGL. Основною метою роботи було створення механізму масштабування та обертання текстури навколо довільної точки на поверхні тривимірного об'єкта, яка визначається користувачем у режимі реального часу. Поставлене завдання виконано повністю.

У процесі реалізації було досліджено принципи роботи UV-координат, механізми шейдерної обробки та особливості застосування афінних перетворень у просторі текстур. Обчислення трансформацій було перенесено на графічний процесор, що забезпечило високу продуктивність і можливість взаємодії без перезавантаження сцени. Запропонований підхід дозволяє змінювати точку pivot за допомогою клавіш A/W/S/D та отримувати коректний візуальний результат як для масштабування, так і для обертання текстури.

Розроблена система демонструє гнучкий і розширюваний підхід до керування текстурними перетвореннями, який може бути використаний у комп'ютерній графіці, ігрових рушіях та навчальних застосунках. Виконана робота підтверджує практичну цінність розглянутої технології та її придатність для подальшого удосконалення.

Лістинг програмного коду:

```
// Light
let t = 0;
let lightPos = [3.0, 0.0, 1.2];
const moveSpeed = 0.4;
addEventListener('keydown', e => {
  const code = e.code;
  const mode = ui.lightMode.value;

  if (mode === 'manual') {
    const { right, up } = getCameraBasis();
```

```
let moved = false;
```

```
if (code === 'KeyW') {  
    lightPos[0] += up[0] * moveSpeed;  
    lightPos[1] += up[1] * moveSpeed;  
    lightPos[2] += up[2] * moveSpeed;  
    moved = true;  
} else if (code === 'KeyS') {  
    lightPos[0] -= up[0] * moveSpeed;  
    lightPos[1] -= up[1] * moveSpeed;  
    lightPos[2] -= up[2] * moveSpeed;  
    moved = true;  
} else if (code === 'KeyA') {  
    lightPos[0] -= right[0] * moveSpeed;  
    lightPos[1] -= right[1] * moveSpeed;  
    lightPos[2] -= right[2] * moveSpeed;  
    moved = true;  
} else if (code === 'KeyD') {  
    lightPos[0] += right[0] * moveSpeed;  
    lightPos[1] += right[1] * moveSpeed;  
    lightPos[2] += right[2] * moveSpeed;  
    moved = true;  
}
```

```
if (moved) {  
    const Rmax = 6.0;  
    const r = Math.hypot(lightPos[0], lightPos[1], lightPos[2]);  
    if (r > Rmax) {  
        lightPos[0] *= Rmax / r;  
        lightPos[1] *= Rmax / r;  
        lightPos[2] *= Rmax / r;  
    }  
}
```

```

    }
}
return;
}

const step = 0.02;
let changed = false;

if (code === 'KeyA') {
    texPivot.u -= step; // вліво по u
    changed = true;
} else if (code === 'KeyD') {
    texPivot.u += step; // вправо по u
    changed = true;
} else if (code === 'KeyW') {
    texPivot.v += step; // вгору по v
    changed = true;
} else if (code === 'KeyS') {
    texPivot.v -= step; // вниз по v
    changed = true;
}

if (changed) {
    texPivot.u = Math.max(-1.0, Math.min(1.0, texPivot.u));
    texPivot.v = Math.max(-1.0, Math.min(1.0, texPivot.v));

    if (ui.texPivotStat) {
        ui.texPivotStat.textContent =
            `Tex pivot (u, v): (${texPivot.u.toFixed(2)}, ${texPivot.v.toFixed(2)})`;
    }

    e.preventDefault();
}

```

```
}  
});
```

```
function draw() {  
  
    if (ui.fillTris.checked) {  
        gl.useProgram(progPhong);  
        gl.uniformMatrix4fv(locP.u_proj, false, proj);  
        gl.uniformMatrix4fv(locP.u_view, false, view);  
        gl.uniformMatrix4fv(locP.u_model, false, modelMat);  
        gl.uniform3fv(locP.u_lightPos, new Float32Array(lightPos));  
        gl.uniform3fv(locP.u_eyePos, new Float32Array(eye));  
        gl.uniform3fv(locP.u_baseColor, new Float32Array([1.0, 1.0, 1.0]));  
        gl.uniform1f(locP.u_specPow, +ui.specPow.value);  
        gl.uniform1i(locP.u_material, +ui.material.value);  
        gl.uniform2f(locP.u_texPivot, texPivot.u, texPivot.v);  
        gl.uniform1f(locP.u_texScale, texScale);  
        gl.uniform1f(locP.u_texAngle, texAngle);  
        gl.activeTexture(gl.TEXTURE0);  
        gl.bindTexture(gl.TEXTURE_2D, texDiffuse);  
        gl.uniform1i(locP.u_diffuseTex, 0);  
        gl.activeTexture(gl.TEXTURE1);  
        gl.bindTexture(gl.TEXTURE_2D, texSpecular);  
        gl.uniform1i(locP.u_specularTex, 1);  
        gl.activeTexture(gl.TEXTURE2);  
        gl.bindTexture(gl.TEXTURE_2D, texNormal);  
        gl.uniform1i(locP.u_normalTex, 2);  
        gl.uniform1i(locP.u_useDiffuseTex, ui.useDiffuseTex.checked ? 1 : 0);  
        gl.uniform1i(locP.u_useSpecularTex, ui.useSpecularTex.checked ? 1 : 0);  
        gl.uniform1i(locP.u_useNormalTex, ui.useNormalTex.checked ? 1 : 0);  
    }  
}
```

```
gl.bindBuffer(gl.ARRAY_BUFFER, model.vbo);
gl.enableVertexAttribArray(locP.a_pos);
gl.vertexAttribPointer(locP.a_pos, 3, gl.FLOAT, false, 0, 0);
gl.bindBuffer(gl.ARRAY_BUFFER, model.nbo);
gl.enableVertexAttribArray(locP.a_norm);
gl.vertexAttribPointer(locP.a_norm, 3, gl.FLOAT, false, 0, 0);
gl.bindBuffer(gl.ARRAY_BUFFER, model.uvbo);
gl.enableVertexAttribArray(locP.a_uv);
gl.vertexAttribPointer(locP.a_uv, 2, gl.FLOAT, false, 0, 0);
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, model.ebo);
gl.drawElements(gl.TRIANGLES, model.ibCount, gl.UNSIGNED_INT, 0);
}

// ... решта рендеру (сфера світла, тінь, wireframe, points) ...
}
```